

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Curso: CE-1103 Algoritmos y Estructuras de Datos I.

Documentación Técnica: Proyecto 1: Cyber Defense Duel

Profesor: MSc. Andrés Vargas

Integrantes:

Ezequiel Bonilla Vega: 2025098474

Luis Fernando Ortiz Mata: 2022206801

Manuel Antonio Rojas Loaiza: 2020072613

18 de abril de 2026

I semestre

Contenido:

1. Introducción	3
2. Arquitectura General del Sistema	3
2.1. Estilo arquitectónico	3
2.2. Módulos principales.....	3
2.3. Flujo general de ejecución	4
2.4. Diagrama de clases UML:	4
3. Protocolo de Comunicación.....	6
3.1. Tecnología y estructura.....	6
3.2. Acciones principales del protocolo	6
3.4. Consideraciones técnicas	9
4. Estructuras de Datos Implementadas Manualmente	10
4.1. Estructuras desarrolladas.....	10
4.2. Uso de estructuras en el sistema	10
4.3. Justificación académica y técnica	10
5. Persistencia con database.json	10
5.1. Modelo de datos	10
5.2. Flujo de lectura y escritura	11
5.3. Concurrencia y consistencia	12
6. Decisiones de Diseño Relevantes (Trade-offs y Justificación)	12
6.1. Uso de mediador central	12
6.2. Comunicación JSON sobre sockets TCP	12
6.3. Persistencia en archivo JSON local.....	12
6.4. Servidor orientado a partida 1 vs 1.....	13
6.5. Pooling de balas con pila	13
7. Limitaciones Actuales y Trabajo Futuro	13
8. Conclusión	14
9. Fuentes Bibliográficas:	14

1. Introducción

Cyber Defense Duel es un videojuego competitivo 1 vs 1 desarrollado en Java, con interfaz gráfica en JavaFX y arquitectura cliente-servidor basada en sockets TCP. El proyecto integra conceptos de programación orientada a objetos, comunicación en red y estructuras de datos implementadas manualmente, alineándose con los objetivos del curso CE-1103.

El sistema permite autenticación de usuarios, selección de avatar y mapa, sincronización del estado entre dos jugadores y persistencia de estadísticas en un archivo JSON local.

2. Arquitectura General del Sistema

2.1. Estilo arquitectónico

- Arquitectura distribuida cliente-servidor.
- Cliente responsable de interfaz y lógica local de partida.
- Servidor responsable de autenticación, coordinación entre jugadores y persistencia.

2.2. Módulos principales

- **Aplicación/UI:**

Manejo de pantallas: login, registro, selección, carga, juego y game over.

- **Lógica de juego:**

Control de jugador, enemigos, balas, colisiones, score y dificultad.

- **Mediación de datos:**

Componente central que desacopla UI, red y gestión de datos del jugador.

- **Red:**

- * Cliente TCP con listener asíncrono.
- * Servidor multihilo con un handler por cliente.

- Persistencia:

Gestor de lectura y escritura sobre base de datos en formato JSON.

- Estructuras de datos:

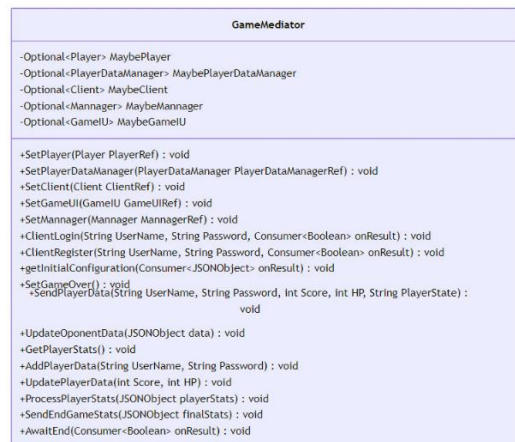
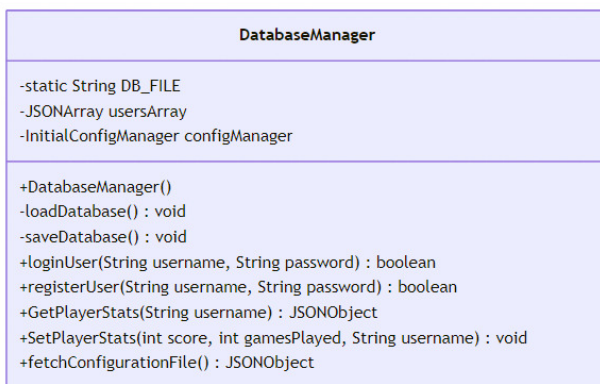
Listas y pilas desarrolladas manualmente para operaciones del juego y pruebas académicas.

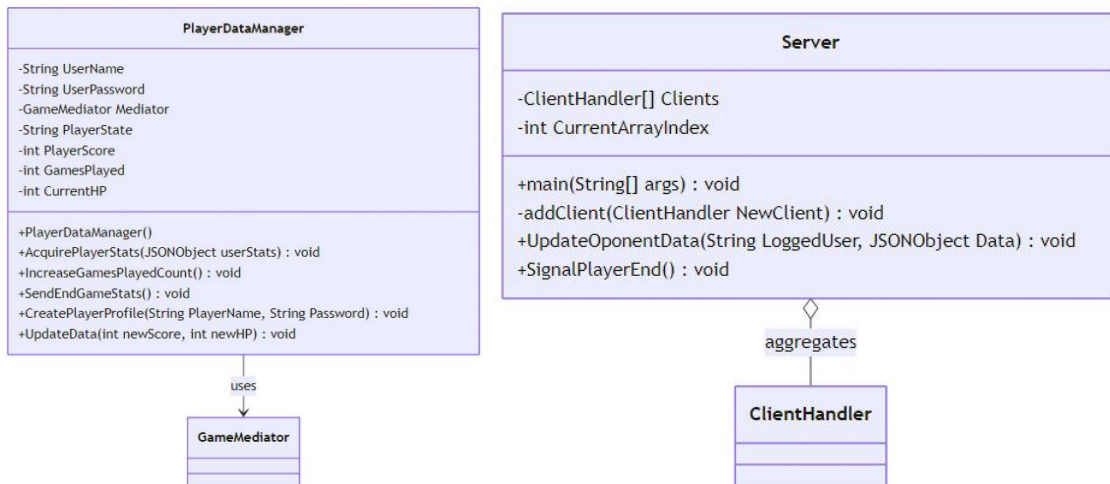
2.3. Flujo general de ejecución

- 1- Usuario inicia cliente e ingresa credenciales.
- 2- Cliente envía solicitud de login o registro.
- 3- Servidor valida y responde.
- 4- Cliente solicita estadísticas y configuración inicial.
- 5- Inicia el loop del juego.
- 6- Cliente envía actualizaciones de estado durante la partida.
- 7- Servidor reenvía información al oponente.
- 8- Al finalizar, cliente envía estadísticas finales.
- 9- Servidor compara puntajes y envía resultado final.

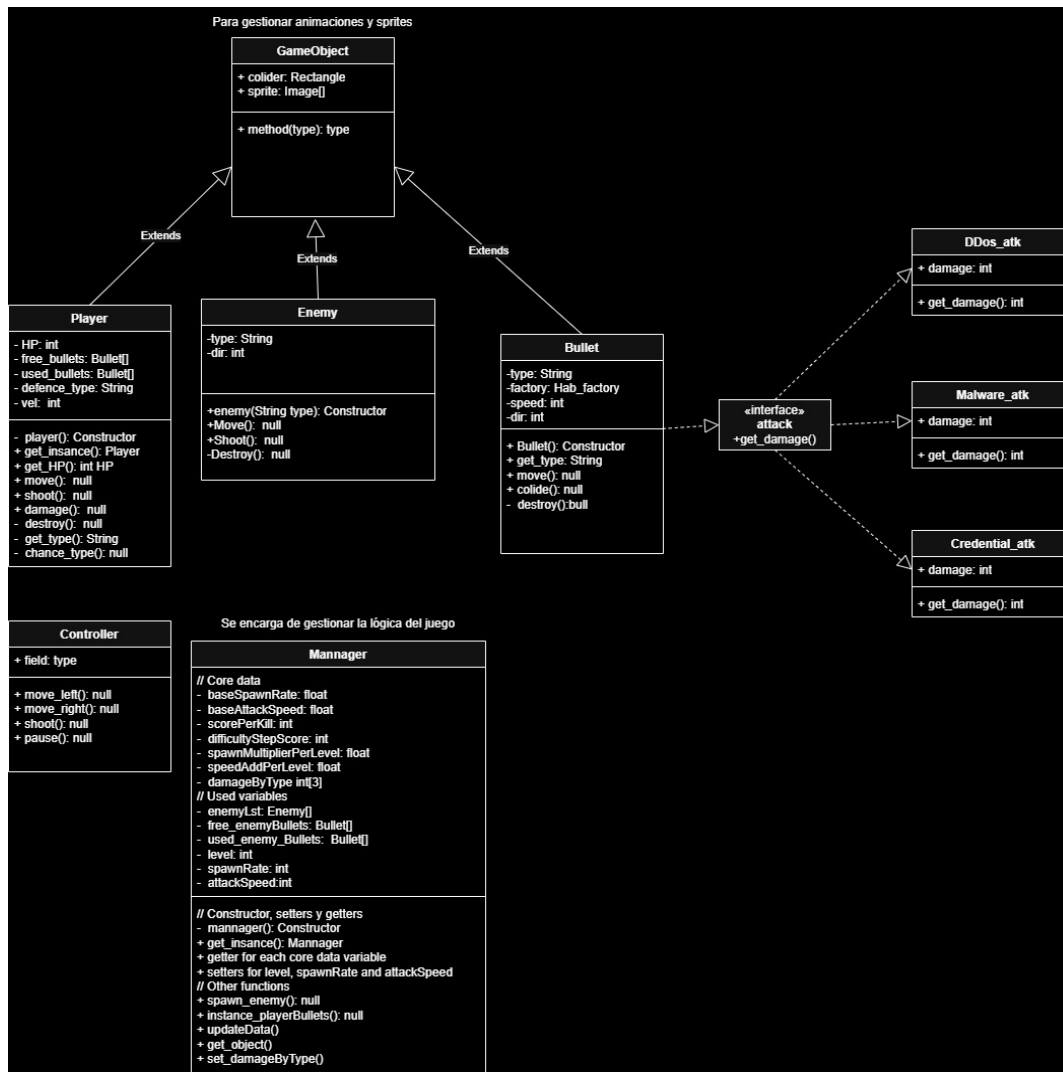
2.4. Diagrama de clases UML:

UML del network (Cliente-servidor):





UML del juego:



3. Protocolo de Comunicación

3.1. Tecnología y estructura

- **Transporte:** TCP.
- **Serialización:** JSON.
- **Patrón:** mensajes request-response y eventos.
- **Campo clave de enrutamiento:** action.

3.2. Acciones principales del protocolo

- Login
- Register
- get_config

- get_stats
- update
- updateOponent
- set_stats
- finalResult

3.3. Ejemplos de mensajes

Solicitud de login:

```
{  
  "action": "login",  
  "userName": "luigi",  
  "password": "pass"  
}
```

Respuesta de login:

```
{  
  "action": "login",  
  "status": "success",  
  "message": "Login Successful"  
}
```

Solicitud de configuración inicial:

```
{  
  "action": "get_config"  
}
```

Respuesta de configuración:

```
{  
  "action": "CONFIG",  
  "initialHp": 100,  
  "baseSpawnRate": 1.0,  
  "baseAttackSpeed": 2.0,  
  "scorePerKill": 10,  
  "difficultyStepScore": 100,  
  "spawnMultiplierPerLevel": 1.15,  
  "speedAddPerLevel": 0.3,  
  "damageByType": {  
    "DDOS": 5,  
    "MALWARE": 8,  
    "CRED": 10  
  }  
}
```

Actualización de estado en partida:

```
{  
  "action": "update",  
  "userName": "luigi",  
  "password": "pass",  
  "score": 120,  
  "hp": 75,  
  "PlayerState": "Active"  
}
```

Mensaje reenviado al oponente:

```
{  
  "action": "updateOpponent",  
  "score": 120,  
  "hp": 75  
}
```

Envío de cierre de partida:

```
{  
  "action": "set_stats",  
  "Score": 140,  
  "GamesPlayed": 6,  
  "PlayerState": "END"  
}
```

Resultado final:

```
{  
  "action": "finalResult",  
  "result": true  
}
```

3.4. Consideraciones técnicas

- El protocolo es simple.
- No existe versión de protocolo ni validación formal de esquemas JSON.
- El manejo de errores incluye acción desconocida y pérdida de conexión.

4. Estructuras de Datos Implementadas Manualmente

4.1. Estructuras desarrolladas

- Lista enlazada simple.
- Lista doblemente enlazada.
- Lista doblemente terminada.
- Lista circular.
- Pila con nodos.
- Cola con nodos.
- Pila y cola con arreglos para comparación de rendimiento.

4.2. Uso de estructuras en el sistema

Enemigos activos:

Lista enlazada que contiene los enemigos que van a estar en pantalla.

Balas enemigas libres:

Pila para las balas que no están en uso.

Balas enemigas activas:

Lista enlazada para las balas que están en uso dentro del juego.

Balas del jugador:

Pila de balas libres y lista de balas en uso, con una implementación igual a las anteriores.

4.3. Justificación académica y técnica

- Se prioriza el uso de estructuras propias para demostrar dominio de algoritmos y estructuras fundamentales.
- Permite observar costo de operaciones de inserción, eliminación y acceso sin dependencia de colecciones estándar.
- Refuerza objetivos del curso en diseño de TAD y análisis de rendimiento.

5. Persistencia con database.json

5.1. Modelo de datos

La persistencia se implementa como un arreglo JSON de usuarios. Cada registro contiene:

- UserName

- Password
- Score
- GamesPlayed

Ejemplo:

```
[  
{  
  "Score": 0,  
  "UserName": "luigi",  
  "Password": "pass",  
  "GamesPlayed": 4  
},  
{  
  "Score": 50,  
  "UserName": "mario",  
  "Password": "pass",  
  "GamesPlayed": 4  
}  
]
```

5.2. Flujo de lectura y escritura

Inicio de servidor:

- Carga completa del archivo a memoria.
- Si no existe, crea base vacía.

Login:

- Recorre usuarios para validar credenciales.

Registro:

- Verifica si el nombre existe.

- Inserta usuario nuevo con score y partidas en cero.
- Guarda el archivo actualizado.

Fin de partida:

- Actualiza score y juegos jugados del usuario.
- Persiste cambios en disco.

5.3. Concurrencia y consistencia

- Operaciones críticas de base se ejecutan con sincronización para evitar escrituras simultáneas.
- La estrategia de persistencia reescribe el archivo completo tras cambios.

6. Decisiones de Diseño Relevantes (Trade-offs y Justificación)

6.1. Uso de mediador central

Decisión:

Introducir un módulo mediador para coordinar UI, red y datos.

Ventaja:

Reduce acoplamiento entre componentes.

Trade-off:

Dependencia de inicialización correcta de referencias internas.

6.2. Comunicación JSON sobre sockets TCP

Decisión:

Protocolo textual con acciones explícitas.

Ventaja:

Facilidad de implementación y depuración.

Trade-off:

Menor robustez frente a protocolos con tipado/contratos formales.

6.3. Persistencia en archivo JSON local

Decisión:

No usar motor de base de datos externo.

Ventaja:

Simplicidad de despliegue y bajo costo.

Trade-off:

Escalabilidad limitada y seguridad reducida.

6.4. Servidor orientado a partida 1 vs 1

Decisión:

Estructura interna fija para dos clientes.

Ventaja:

Implementación simple para alcance del proyecto.

Trade-off:

Difícil escalar a múltiples salas o emparejamiento dinámico.

6.5. Pooling de balas con pila

Decisión:

Reusar instancias en lugar de crear/destruir constantemente.

Ventaja:

Menor sobrecarga de memoria y mejor estabilidad en runtime.

Trade-off:

Mayor complejidad de manejo de ciclo de vida de objetos.

7. Limitaciones Actuales y Trabajo Futuro

Seguridad:

- Contraseñas almacenadas en texto plano.
- Tráfico sin cifrado.

Escalabilidad:

- Estructura de servidor acotada a dos clientes activos.

Robustez:

- Falta validación formal de mensajes JSON.

Mejoras propuestas:

- Hash de contraseñas y cifrado de transporte.
- Definición de contrato de mensajes con validación.
- Gestión de salas y emparejamiento escalable.
- Persistencia con base de datos real.
- Métricas de latencia y tolerancia a reconexión.

8. Conclusión

Cyber Defense Duel cumple con los objetivos formativos de CE-1103 al integrar:

- Arquitectura cliente-servidor funcional.
- Comunicación en red mediante mensajes JSON.
- Implementación y aplicación de estructuras de datos manuales.
- Persistencia básica de estado del usuario.
- El diseño prioriza claridad y valor académico, y deja una base sólida para evolucionar hacia un sistema más seguro, robusto y escalable.

9. Fuentes Bibliográficas:

OpenFX (s.f). Java FX. <https://openjfx.io>

Google. (2026) Gemini (Version Gemini 2.5 Flash) [Modelo de lenguaje de gran tamaño]. <https://gemini.google.com/app>

GitHub Copilot · Your AI pair programmer. (2026). GitHub.
https://github.com/features/copilot?utm_source=copilot.com

Learn Java - dev.java. (s. f.). Dev.java: The Destination For Java Developers.
<https://dev.java/learn/>

JDK 25 Documentation - Home. (2026, 17 marzo). Oracle Help Center.
<https://docs.oracle.com/en/java/javase/25/>

Munjal, H. (2026, 25 marzo). *8 basic data structures plus a guide to algorithms*.
GoDaddy Resources - India. <https://www.godaddy.com/resources/in/web-pro-in/8-basic-data-structures-every-programmer-should-know>